A04:2025 Cryptographic Failures

A04:2025 Cryptographic Failures

為什麼它排在第四名？

現狀變更（Status Change）

- 雖然排名從 **#2 下降至 #4**，但「密碼學失效」仍然是系統崩潰的核心原因。
- 這通常源於：
  - **缺乏加密**
  - **使用強度不足的加密**
  - **或金鑰洩漏**

不再是效能問題（Performance is no excuse）

- 現代 CPU 已內建 **AES 加速指令集**。
- TLS 憑證管理已透過 **Let’s Encrypt** 和雲端服務自動化。

- 捨棄加密不再有藉口。

A04:2025 Cryptographic Failures

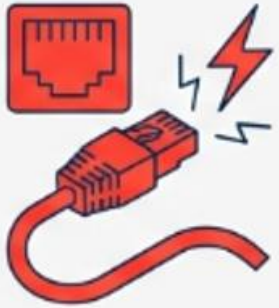
程式碼中的危險訊號（Red Flags）

過時的演算法

還在用 **MD5**、**SHA1**、**RC4**、**DES** 嗎？
這些已被證實不安全。
是否使用了 **ECB 模式**（**Electronic Codebook**），而非更安全的認證加密模式？

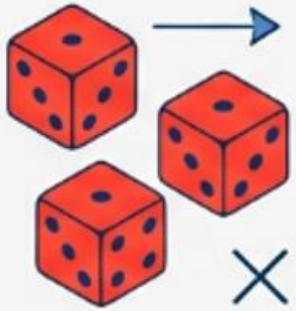
金鑰管理失當

金鑰是否直接寫死（Hardcoded）
在 **config.php** 或 **Git 儲存庫** 中？
是否缺乏金鑰輪替機制（Key Rotation）？



傳輸層漏洞

是否缺少 **HTTP Headers**（**如 HSTS**）？
是否未驗證伺服器憑證及其信任鏈？

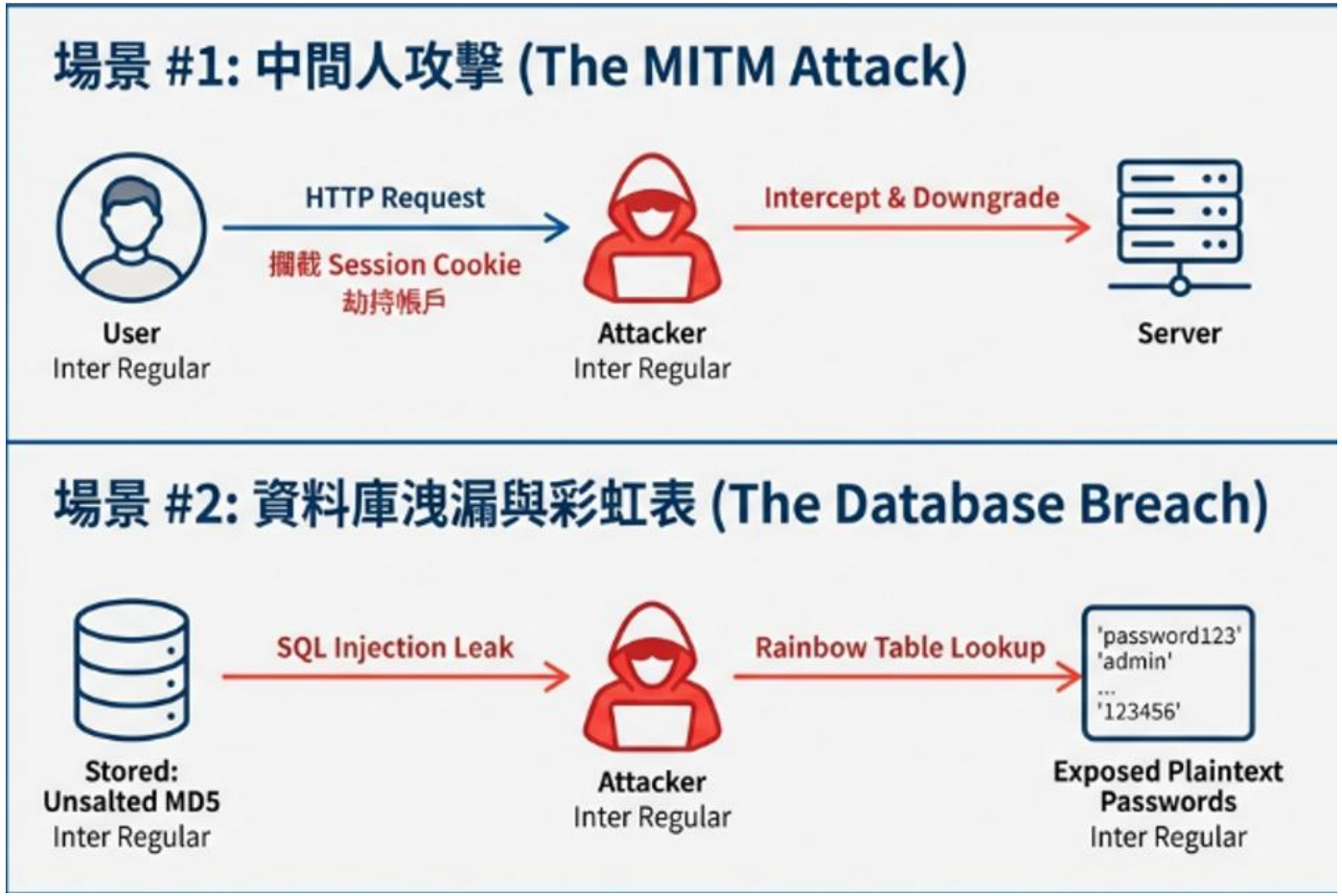


隨機數陷阱

是否在需要加密強度的場景使用了一般的隨機函數（如 **rand()**、**mt_rand()**）？

A04:2025 Cryptographic Failures

攻擊場景解析（Attack Scenarios）

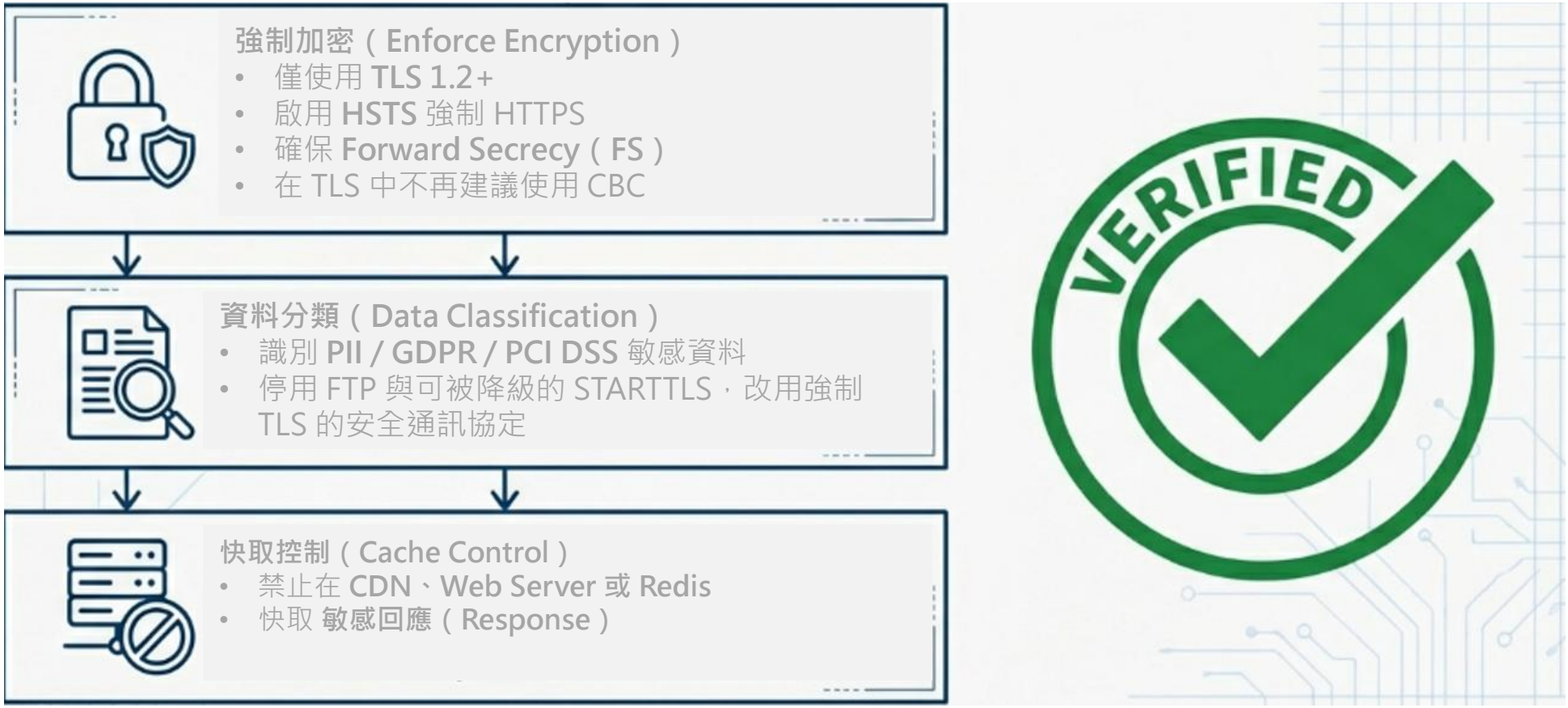


漏洞：應用程式未強制全站 TLS。
影響：攻擊者將 HTTPS 降級為 HTTP，竊取使用者憑證。

漏洞：使用無鹽（Unsalted）或簡單雜湊儲存密碼。
影響：攻擊者利用預算好的彩虹表，瞬間反查出原始密碼。

A04:2025 Cryptographic Failures

- 防禦策略 I：傳輸與架構（Transport & Architecture）
- Defense Layers



A04:2025 Cryptographic Failures

防禦策略 II：密碼儲存與金鑰（Password Storage & Keys）

VULNERABLE

絕對禁止 MD5, SHA1, Base64

```
$password = $_POST['password'];
// BAD: Unsalted, fast hash
$hash = md5($password);
// BAD: Hardcoded key
$key = 'my_secret_key_123';
```

SECURE

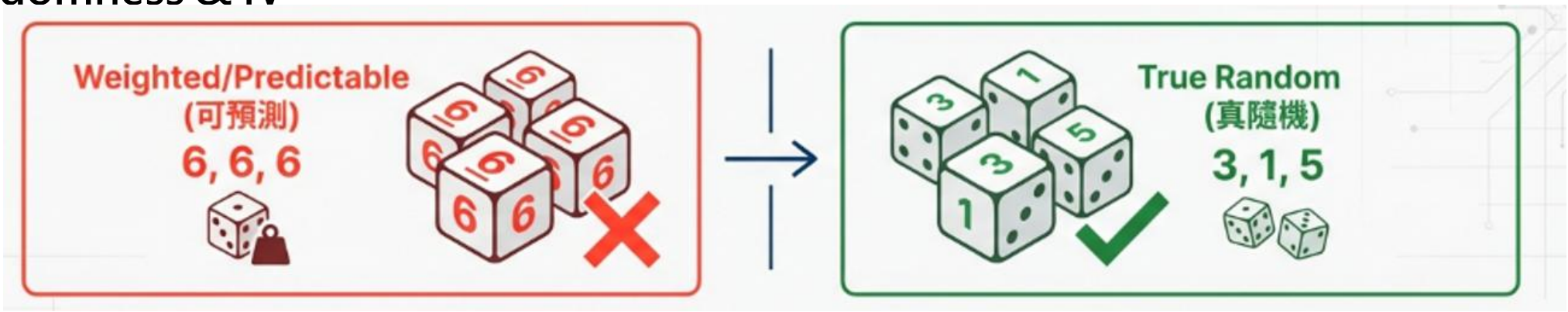
使用自適應雜湊（Adaptive Hashing）
Argon2 / Bcrypt

```
$password = $_POST['password'];
// GOOD: Built-in salt, cost factor
$hash = password_hash($password, PASSWORD_ARGON2ID);
// GOOD: Key from ENV
$key = getenv('ENCRYPTION_KEY');
```

加密實作建議：總是優先使用 Authenticated Encryption（如 GCM 模式）而非單純加密。

A04:2025 Cryptographic Failures

防禦策略 III：隨機數與初始化向量
Randomness & IV



隨機數生成器（PRNG）

錯誤（Fail）
- 使用 rand() 或 mt_rand() 生成 Token
- 這是 **可預測的**

正確（Pass）
- 使用 **CSPRNG**
  （Cryptographically Secure Pseudo-Random Number Generator

```
php
$token = bin2hex(random_bytes(32));
$int   = random_int(1, 100);
```

初始化向量（IV）

IV 必須針對加密模式正確選擇

關鍵規則

對於固定金鑰

IV 絕對不可重複使用（Never Reuse）

實務建議

大多數現代 API 不需要開發者手動 Seed

避免因手動 Seed 導致的熵不足

A04:2025 Cryptographic Failures

開發者立即檢核清單（Developer Checklist）

1. 所有傳輸是否已強制 TLS 1.2+？
2. 是否已移除 Git 中的所有金鑰與機密資訊？
3. 密碼儲存是否已遷移至 Argon2 / Bcrypt？
4. 隨機數生成是否已改用 random_int()？

未來展望：後量子密碼學 (PQC)

因應量子計算威脅，高風險系統需在 2030 年底前準備好 PQC 轉型 (參考 NIST/ENISA 標準)。

參考資源：OWASP ASVS (V11, V12, V14)，OWASP Cheat Sheet Series.

A04:2025 Cryptographic Failures

剖析 JSON Web Token（JWT）：結構與最大的安全迷思

理解 A04：Insecure Design 的起點

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpvag4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SfKxwRJSMeKKF2QT4fvwpMeJf36Pk6yJV_adQssw5c
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC9.eyJzdWIiOiIxMjM0NTY3ODkwIiwiaWF0IjoxNTE2MjM5MDIyfQ.SfIKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Header (標頭)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload (負載)

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "role": "admin"
}
```

Signature (簽章)

```
HMACSHA256(
  base64UrlEncode(header)
  + "." +
  base64UrlEncode(payload),
  secret)
```

安全警示: Base64 編碼 ≠ 加密

JWT 預設僅是「簽章」而非「加密」。任何攔截到 Token 的人，都能直接解碼並讀取 Payload 內容。切勿將機敏資訊直接放入 Payload。

定義

JWT 是一種 **緊湊、可嵌入 URL 的 Token 格式**，用於在兩個實體間傳遞宣告（Claims），是 **身分驗證與授權的核心機制**。

結構拆解

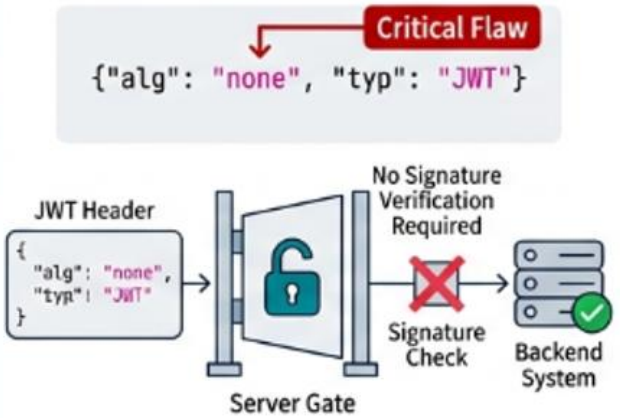
- Header（標頭）：包含 Token 的中繼資訊與簽章演算法（如 **HS256、RS256**）。
- Payload（負載）：存放使用者資料（**User ID、Role、Exp**）。
- Signature（簽章）：使用密鑰對前兩段進行簽章，確保資料完整性（**Integrity**）。

A04:2025 Cryptographic Failures

設計層面的崩壞：破壞 Token 的完整性

當後端盲目信任前端傳來的 Header 資訊，
將導致嚴重的 **OWASP A04（Insecure Design）** 風險。

None 演算法攻擊

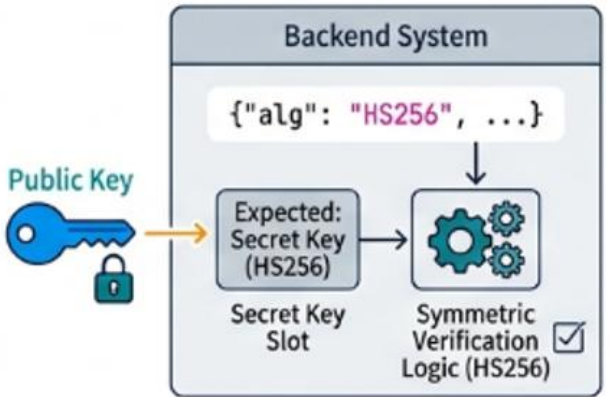


機制：
JWT 規範允許 alg: none代表「不需要簽章」
攻擊：
攻擊者將 Header 改為 none 並移除簽章
若後端未強制檢查，系統將接受任意竄改的 Payload（例如 role=user → admin）
防禦：後端驗證邏輯必須完全禁止 none 演算法

弱密鑰暴力破解

機制：
使用過短或常見字串作為 HMAC 密鑰
攻擊：
攻擊者使用 Hashcat 等工具，本地暴力破解，
一旦算出密鑰，即可隨意偽造 Token

演算法混淆



機制：
某些實作允許在驗證時動態切換 HS（對稱）與 RS（非對稱）
攻擊：
攻擊者將 Header 改為 HS256
並使用後端的 Public Key 當作對稱密鑰簽章
系統若錯誤使用對稱驗證邏輯
驗證會通過

A04:2025 Cryptographic Failures

實作細節的陷阱：洩漏、重放與配置錯誤

即使加密機制完美，
錯誤的傳輸與儲存方式仍會導致 Token 遭竊或濫用。

錯誤訊息洩漏（Verbose Error Messages）

```
HTTP 500 Error
{
  "error": "Signature verification failed",
  "expected": "d29... (leaked signature)" }
```

部分 API 在 Debug 模式下會回傳詳細錯誤，攻擊者可傳送惡意 Token，誘使系統在錯誤訊息中回傳預期的簽章或敏感資料

Token 暴露於 URL

https://api.example.com/data?token=eyJhbGciOi...

若將 Token 放在 URL 參數傳遞
它將留存在瀏覽器歷史紀錄，Proxy Logs，Referrer Header

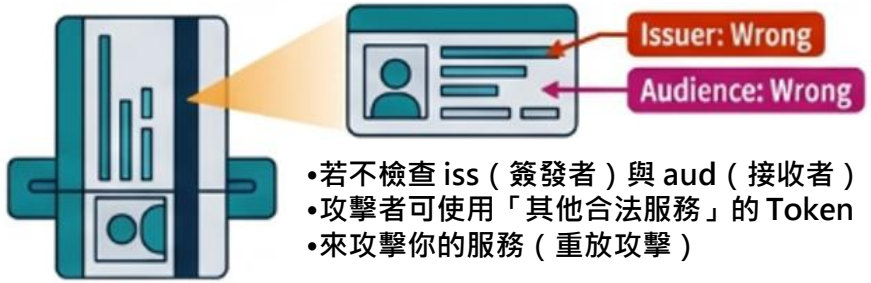
Fix：必須使用 Authorization: Bearer HTTP Header 傳遞

缺乏生命週期管理（No Expiration）

- 未設定 exp（Expiration Time）
- Token 一旦被竊
- 攻擊者將擁有永久存取權

Fix：設定合理過期時間建立 Refresh Token 機制

未驗證簽發者與接收者



- 若不檢查 iss（簽發者）與 aud（接收者）
- 攻擊者可使用「其他合法服務」的 Token
- 來攻擊你的服務（重放攻擊）

A04:2025 Cryptographic Failures

隱形的威脅：時序攻擊（Timing Attack）

攻擊機制

- 許多函式庫在驗證簽章時，一旦發現字元不符即立即停止比對並回傳失敗。
- 攻擊者可測量 API 回應時間的微小差異，逐一猜測出正確簽章的每一個 Byte。

```
if ($user_hmac == $real_hmac)
```

```
if (hash_equals($real_hmac, $user_hmac))
```

數據實證

- 根據研究，在每秒約 **55 次請求** 的速率下，
- 攻擊者可於 **22 小時內**
- 推算出合法的 **HMAC 簽章**。

防禦策略

- 使用 **Constant-time comparison**
- （如 PHP 的 hash_equals）
- 無論比對結果如何，耗時皆相同，消除時間差異。

A04:2025 Cryptographic Failures

防禦縱深：構建安全的 JWT 實作清單

針對 OWASP A04 弱點的完整防禦策略

Flight Check

加密與演算法（Algorithm Hardening）	資料驗證與儲存（Validation & Storage）
☐ 強密鑰：使用長度足夠且隱機性高的密鑰，抵禦暴力破解。	☐ 完整性檢查：嚴格驗證 iss（簽發者）與 aud（接收者）以防止 Token 混用。
☐ 鎖定演算法：後端強制指定單一簽章演算法，禁止 none，不接受前端指定的 alg。	☐ 生命週期：設定合理的 exp 時間，並實作 Token 撤銷機制（Revocation）。
☐ 安全比對：驗證簽章時務必使用 Constant-time comparison 防止時序攻擊。	☐ 傳輸安全：僅透過 HTTP Header 傳輸，並確保全程使用 HTTPS。
進階選項（Advanced）	
☐ 若 Payload 包含敏感個資，請改用 JWE（JSON Web Encryption）進行加密。	☐
☐ 定期更新 JWT 函式庫以修補已知漏洞。	☐

A04:2025 Cryptographic Failures

A04:2025 Cryptographic Failures

錯誤示範：你的程式碼是否這樣寫？

看似正常的 JWT 簽署流程，其實隱藏嚴重漏洞

```php
$this->signKey = (string) ($_ENV['JWT_SECRET'] ?? '');

$claims = [
    'iss' => $this->expectedIss,
    'aud' => $this->expectedAud,
    'iat' => $now,
    'jti' => $now,
    'id' => $user['id'],
];

return JWT::encode($claims, $this->signKey, 'HS256');
```

A04:2025 Cryptographic Failures

為何錯誤？命中 A04 關鍵風險

```
($_ENV['JWT_SECRET'] ?? '')
```

區塊一：弱金鑰管理（Key Management Error）
允許空字串 fallback 等同於允許「弱金鑰」。
一旦環境變數未設定，系統不會報錯，反而使用極弱的空值加密。
違反「Insufficiently strong cryptography」。

```
'jti' => $now
```

區塊二：熵不足（Insufficient Entropy）
time() 是可預測數值，並非唯一識別碼。
這導致 Token 容易遭受重放攻擊（Replay Attacks），
且不具備足夠的隨機性。

```
'jti' => bin2hex(random_bytes(16)),
```

```
missing checks...
```

區塊三：缺乏完整性檢查
原始程式碼未檢查 iss 或 aud 是否存在，
直接信任輸入值

A04:2025 Cryptographic Failures

如何修正：符合 A04 標準的實作

對

對

```php
// 1. 強制檢查金鑰強度
if ($this->signKey === '' || strlen($this->signKey) < 32) {
    throw new \RuntimeException('JWT_SECRET missing or too weak.');
}

$claims = [
    // ...
    'nbf' => $now, // 2. 明確生效時間
    'jti' => bin2hex(random_bytes(16)), // 3. 高熵隨機值
    // ...
];
```

A04:2025 Cryptographic Failures

修正重點解析

強制強金鑰

不再依賴預設值。
若 JWT_SECRET 缺失或長度不足 32 bytes，
直接拋出例外（Throw Exception），
阻止系統在不安全狀態下運作。

不可預測隨機值

使用 random_bytes() 替代 time()。
確保 jti（Token ID）具有高熵值，
防止碰撞與猜測，
有效降低重放攻擊風險。

完整性驗證

在解碼階段（Decode），
明確檢查 iss（發行者）、aud（接收者）、
exp（過期時間）與 sub（主旨）是否存在，
而非假設它們一定有值。

A04:2025 Cryptographic Failures

核心觀念總結

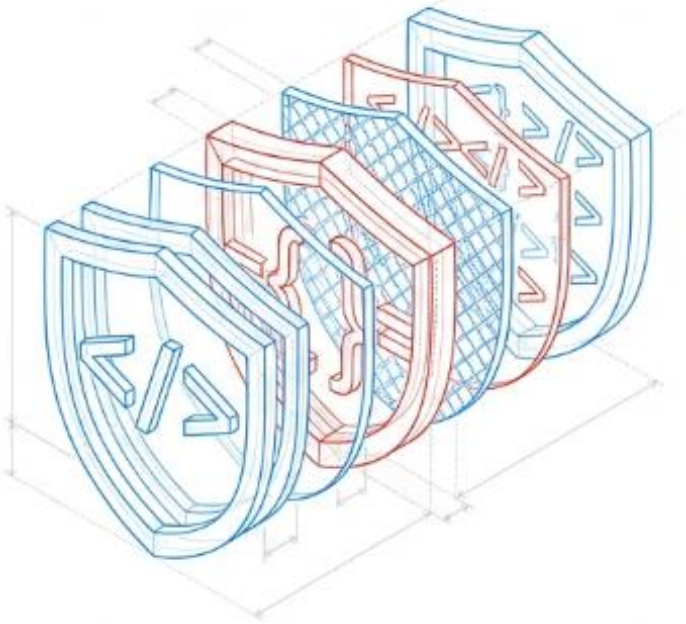
「用對演算法（如 HS256）不代表安全，
金鑰管理與隨機性才是重點。」

禁止預設空值：
金鑰必須強制檢查長度與複雜度
使用高熵隨機源：
使用 random_bytes 等加密安全函數
明確檢查必要欄位：
不要信任輸入，驗證所有 Claims 的存在性

A05:2025 Injection

網頁安全實戰：從 XSS 到 CSP 的縱深防禦

為研發人員打造的上下文感知防禦指南



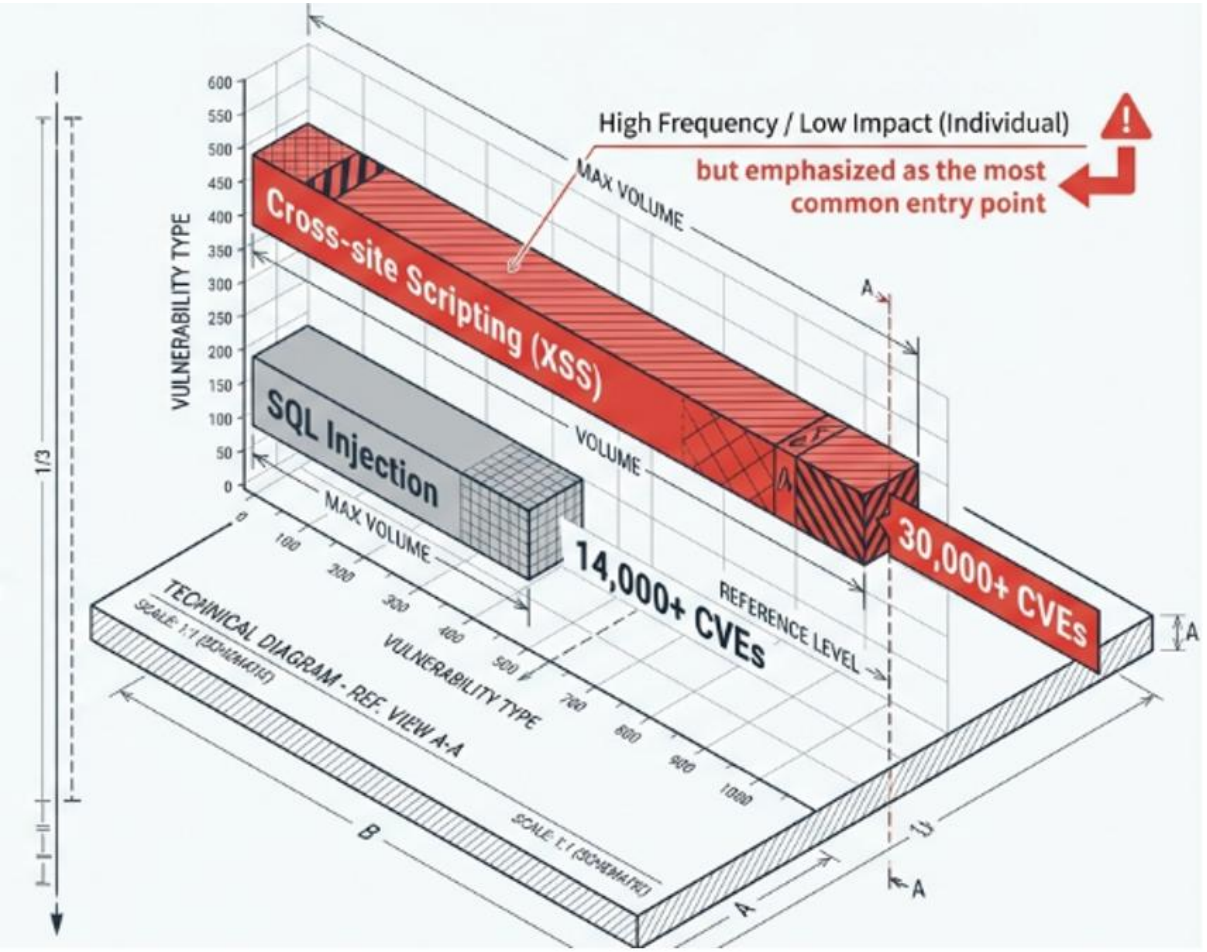
TYPE: TECHNICAL BRIEFING | LEVEL: ENGINEERING / RD | STATUS: V1.0

A05:2025 Injection

為何我們仍需談論 Injection？

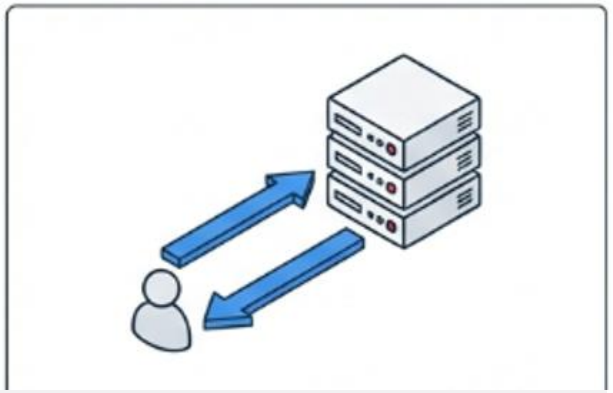
資料來源：OWASP Top 10 A05:2025

- Ranking：#5
  （風險排名）
- Coverage：#1
  （測試覆蓋率）
- 核心問題：應用程式盲目信任使用者輸入
- 缺乏「上下文感知」的轉譯處理



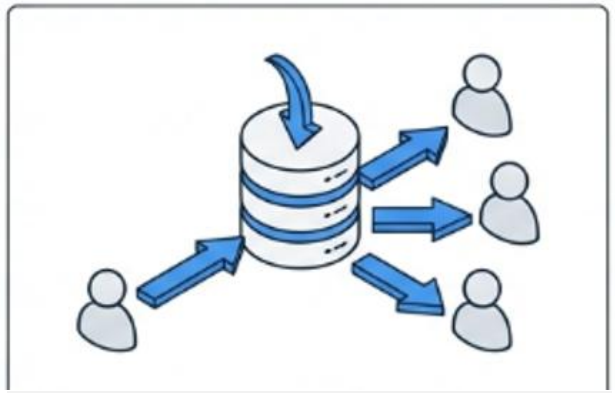
A05:2025 Injection

XSS 攻擊的三種主要型態



Reflected XSS（反射型）
Payload 立即由伺服器回應。
常見於搜尋結果或錯誤訊息。

```
?q=<script>...
```



Stored XSS（儲存型）
Payload 進入資料庫或檔案系統。
影響所有瀏覽該內容的使用者。

```
Saved: <script>...
```

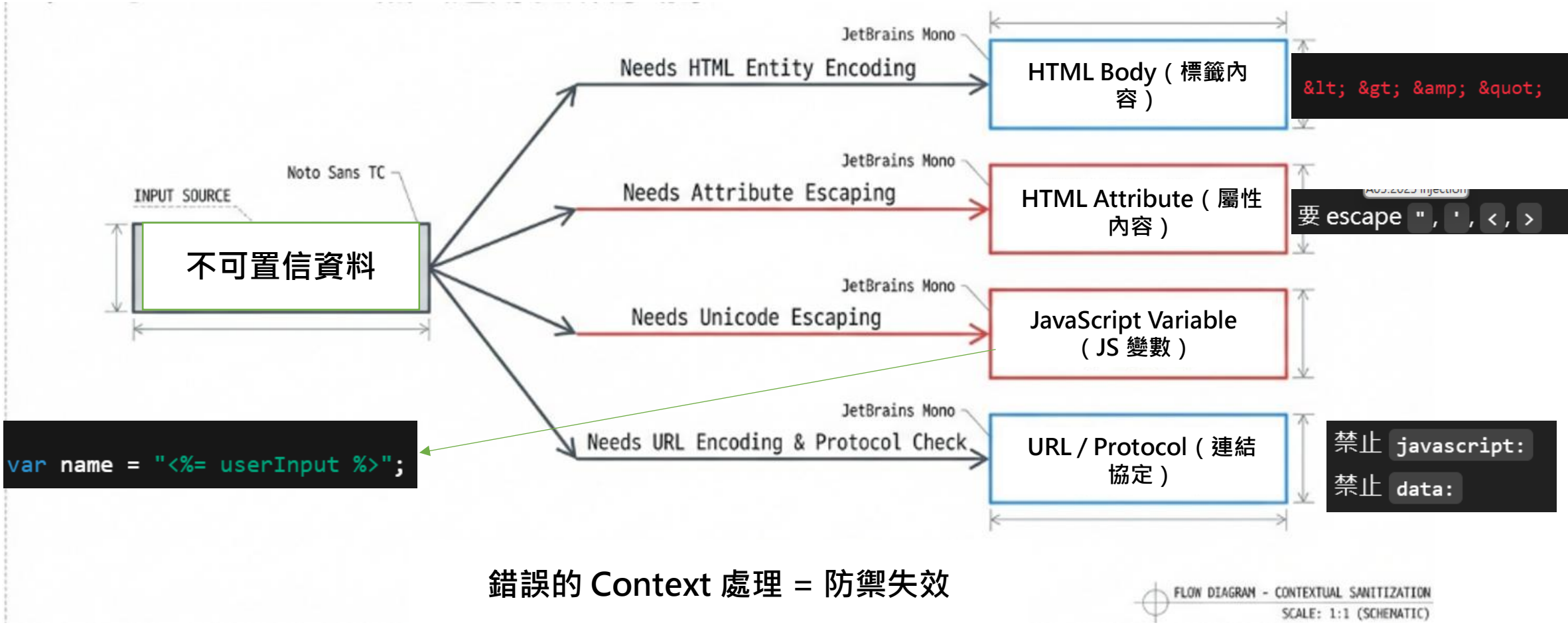
File-based XSS（檔案型）
惡意腳本嵌入於 SVG 或 XML 檔案中。
當瀏覽器直接渲染檔案時觸發。

```
<svg onload="..">
```

A05:2025 Injection

核心觀念：Context（上下文）決定一切

單一的 Sanitization 無法適用於所有場景



錯誤的 Context 處理 = 防禦失效

A05:2025 Injection

Context 1: HTML Body vs. Attributes

相對安全與陷阱的比較分析

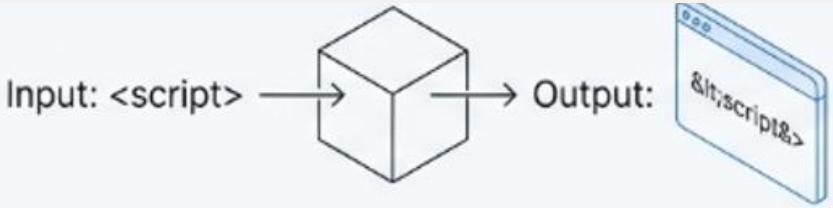
```
&lt;script&gt;alert(1)&lt;/script&gt;
```

Scenario A: HTML Body（相對安全）

Code:

```
<p><?php echo $var; ?></p>
```

•防禦：htmlspecialchars($var) 通常足夠。
•瀏覽器將 < 視為文字而非標籤開頭。



Scenario B: Attributes（陷阱）

Code:

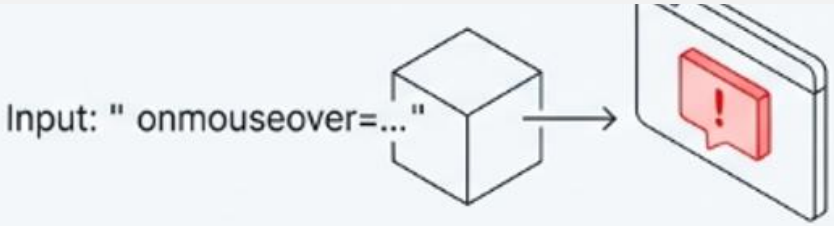
```
<p class="<?php echo $var; ?>"></p>
```

風險：攻擊者可使用引號 " 關閉屬性，並注入新的 Event Handler。
單純 HTML Entity 編碼在某些屬性中可能不足。

攻擊向量視覺化：

Input: ⟶ " onmouseover="alert(1)"

Result: ⟶



A05:2025 Injection

Context 2: JavaScript 字串（The Danger Zone）

在 <script> 標籤內，HTML 編碼無效

```
<script>
  var comment = "$var";
</script>
```

Input Point

Attack Breakout

Attack 1: String Breakout

Input: "; alert(1); //

Result Code:

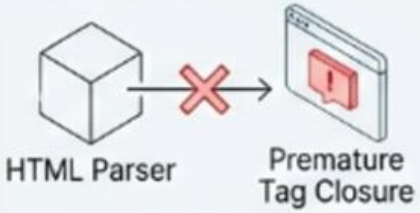
```
<script>
  var comment = ""; alert(1); //";
</script>
```

[redacted]

Attack 2: Script Closing Attack

Input: </script><script>alert(1)</script>

HTML 解析器會在 JS 執行前先看到 </script>，導致提前閉合標籤



正確防禦使用 Unicode Escapes（例如 \u0022 代表 "）

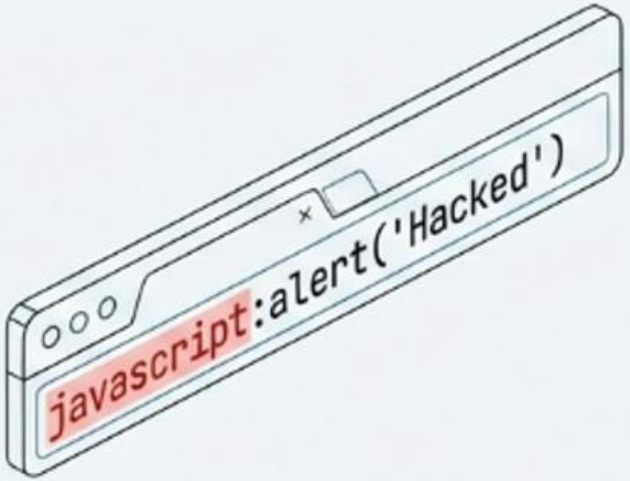
A05:2025 Injection

Context 3: URL 與協定

Vulnerable Example

[redacted]

```
<a href="$var">User Profile</a>
```



瀏覽器會將 javascript: 視為可執行代碼，而非連結。

Defense Logic

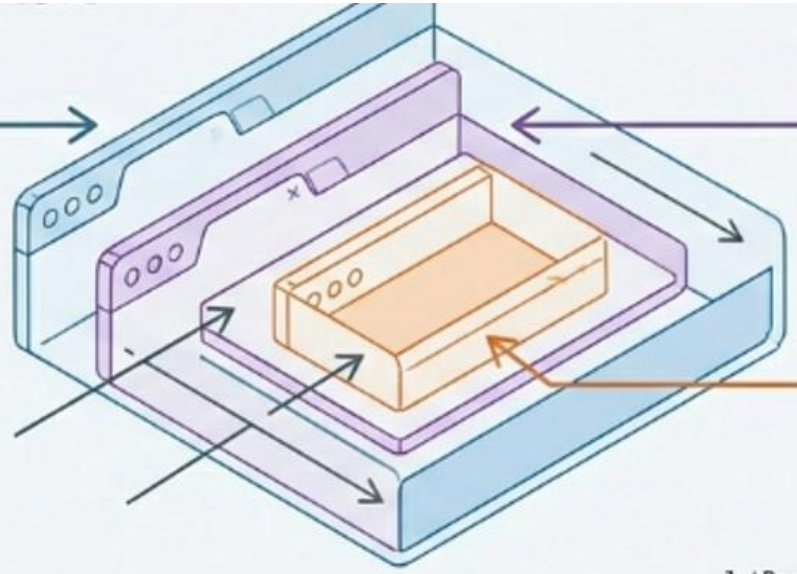
[redacted]

```
$scheme = parse_url($var, PHP_URL_SCHEME);
if ($scheme === 'http' ||
    $scheme === 'https') {
  // Safe to output
  echo '<a href="" .
     htmlspecialchars($var) . '">...</a>';
} else {
  // Block / Error
}
```

驗證重點：不僅是編碼內容，更要白名單驗證協定

A05:2025 Injection

多重 Context 的陷阱



JetBrains Mono

```
<a href="javascript:change('$var')">Click</a>
```

防禦需求：三層編碼（Three-Layer Encoding）

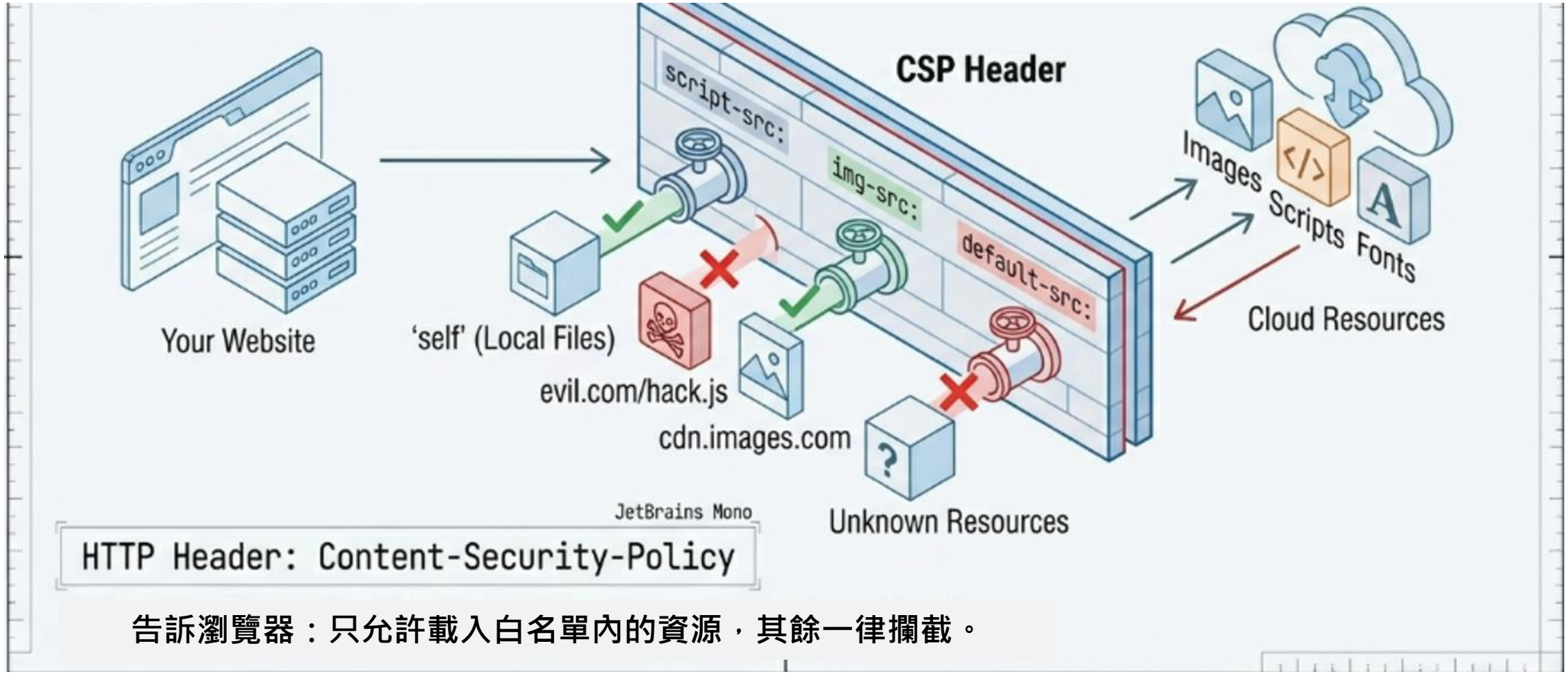
1.JS Encode (\uXXXX)　　2. URL Encode (%XX)　　3.HTML Entity Encode (&...;)

手動處理極易出錯，我們需要更強的安全網。

A05:2025 Injection

安全網：Content Security Policy (CSP)

瀏覽器的白名單機制



告訴瀏覽器：只允許載入白名單內的資源，其餘一律攔截。

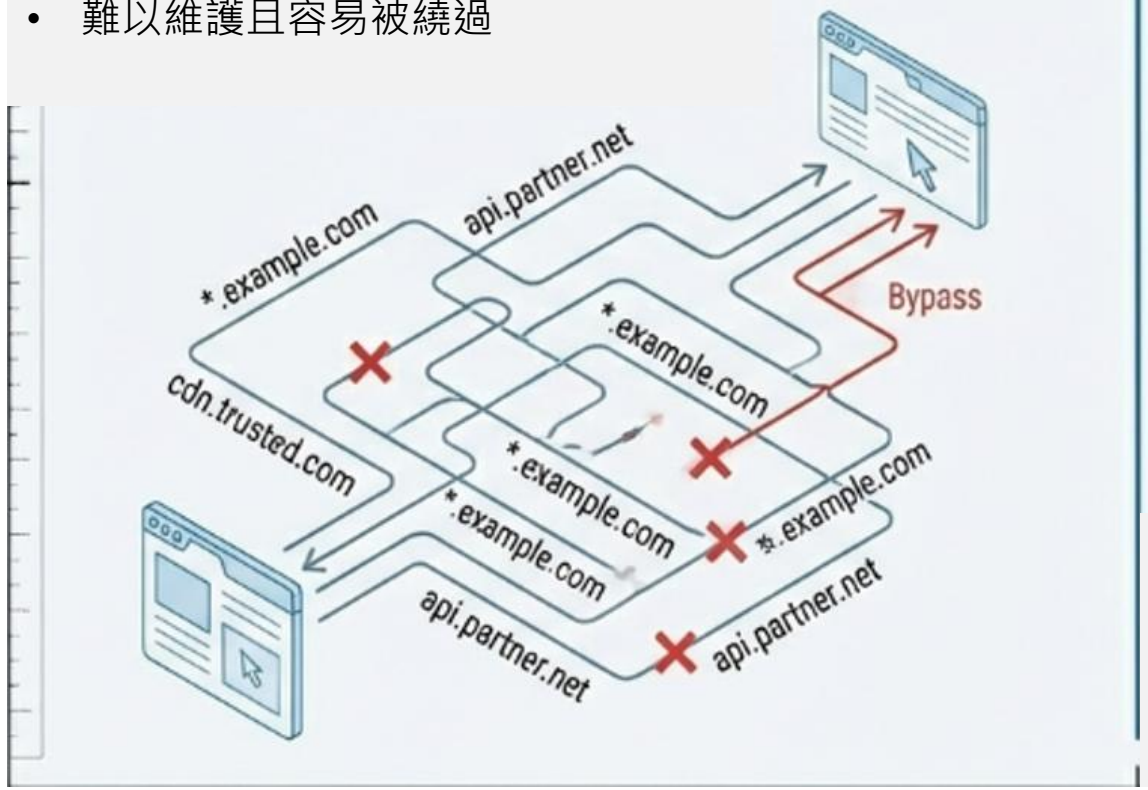
A05:2025 Injection

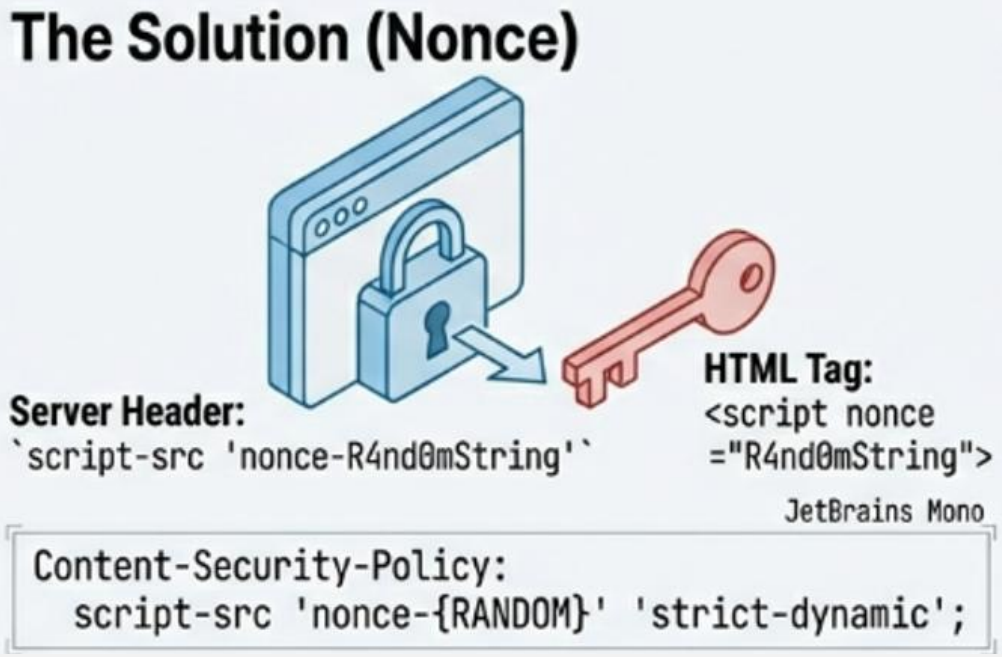
現代 CSP 策略：Nonce 與 Strict-Dynamic

從「信任網域」轉向「信任身份」

The Problem

- Domain Whitelisting（網域白名單）
- 難以維護且容易被繞過





Nonce（Number used ONCE）

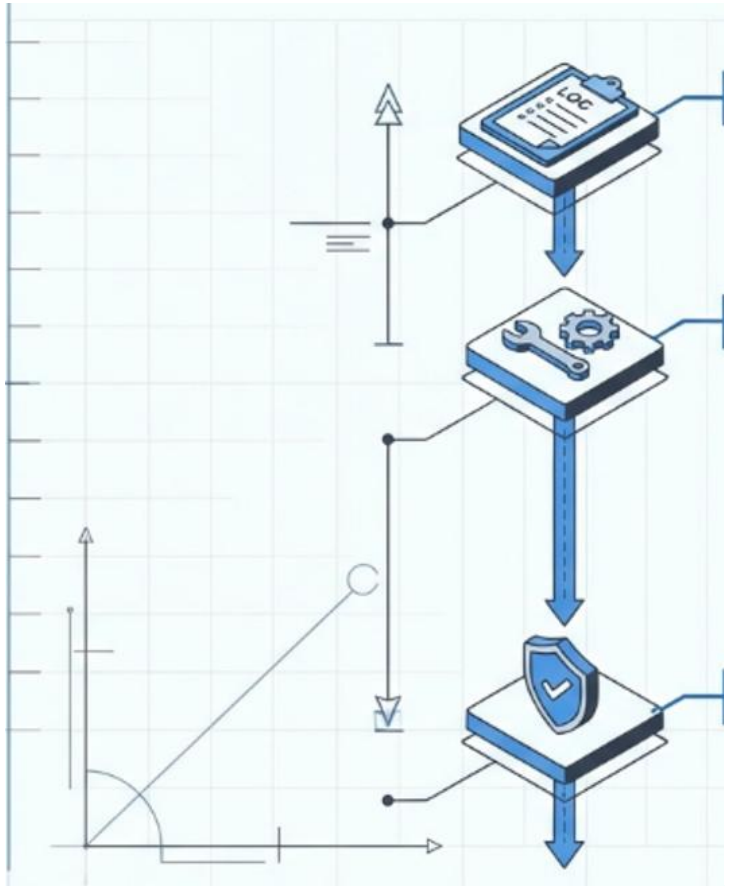
- 伺服器每次請求都產生一組隨機字串
- 只有帶正確 Nonce 的 script 才能執行

strict-dynamic

- 允許「被信任的 script」再動態載入其他相依 script

A05:2025 Injection

CSP 實作落地的建議



Audit & Report Only

Header：Content-Security-Policy-Report-Only

- 不阻擋流量，僅收集違規 Log。
- 確認現有程式碼行為。

Refactor（重構不安全代碼）

- 移除 unsafe-inline 與 eval()
- 轉移 Inline Event Handlers

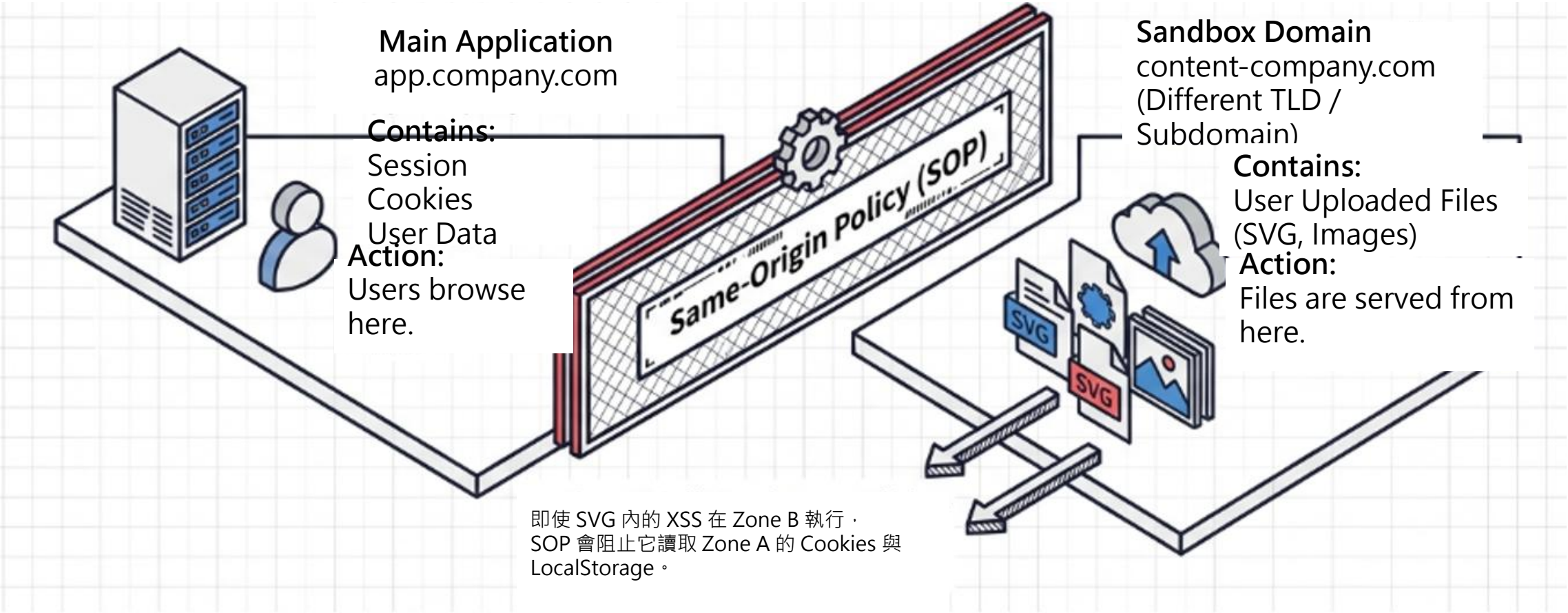
Enforce（強制執行）

- 切換至 Content-Security-Policy（Blocking mode）
- 持續監控 Reporting API

A05:2025 Injection

架構層級防禦：檔案上傳隔離

防止 File-based XSS（SVG / XML）的最佳實踐



A05:2025 Injection

總結：RD 的安全檢查清單

1. Input Validation（輸入驗證）

嚴格驗證型別、格式與長度（Strict Typing）。

2. Context-Aware Output Encoding（依據上下文編碼）

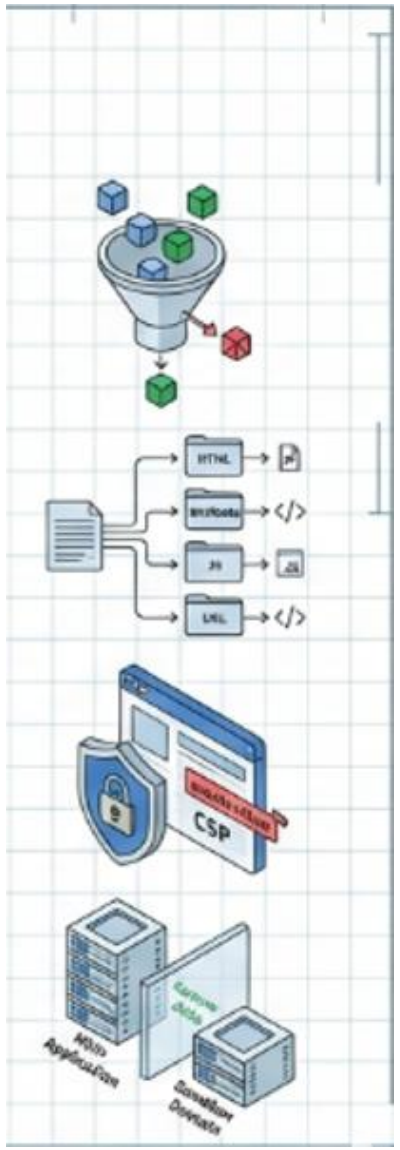
HTML vs Attribute vs JS vs URL

選擇正確的編碼函式。

3. Deploy Strict CSP（部署 CSP）

使用 Nonce-based 策略

拒絕 unsafe-inline。

4. Architecture Isolation（架構隔離）

將高風險的使用者內容隔離至獨立網域。